ELI — Common Errors and Fixes

ELI v2.1.23

July 2026

Contents

ELI — Common Errors & Fixes

> **Updated for v2.1.23 (July 2026).** The primary install path is now the prebuilt, CI-launch-tested installers on GitHub Releases (Windows Setup.exe, macOS dmg, Linux AppImage) with first-boot GPU (CUDA/Vulkan/Metal) and starter-model offers; data lives in a per-user `ELI_v2` folder that survives upgrades. Source installs below remain fully supported.

A quick reference for the gremlins that actually come up when running ELI. Each one is symptom → cause → the exact fix. Add new ones as they surface.

Clicking a link opens no browser page

Symptom: the server prints the URLs fine and copy-paste works, but Ctrl/right-click → "Open Link" opens nothing. **Cause:** broken **snap Firefox** — a `mesa-2404` auto-refresh leaves Firefox's GPU content-mount stale (`/snap/firefox/…/gpu-2404-provider-wrapper` goes missing), so *launching* a fresh Firefox fails silently while an already-open window still works. **Fix:**

```
pkill -9 firefox
sudo snap disconnect firefox:gpu-2404
sudo snap connect firefox:gpu-2404 mesa-2404:gpu-2404
# if it's still stuck:
sudo snap refresh firefox
sudo snap refresh mesa-2404
```

Reopen Firefox once, then relaunch the server — clicks (and the auto-open) work again. *(Nothing wrong with ELI — the URLs and server are fine; it's the snap.)*

Wrong / tiny model loads instead of your usual one

Symptom: ELI boots on a small model (e.g. tinyllama-1B) even though settings point at your 35B A3B. **Cause:** a stray `ELI_GGUF_MODEL_PATH` / `ELI_MODEL_PATH` env var in your shell **overrides config** (it wins over everything). **Fix:**

```
echo $ELI_GGUF_MODEL_PATH        # if this prints a model path, that's the culprit
unset ELI_GGUF_MODEL_PATH ELI_MODEL_PATH
```

Or just switch live from the dashboard: **Settings → Model**.

Server won't start / "address already in use"

Cause: a previous instance is still bound to the port. **Fix:**

```
pkill -f "api/server.py"      # or find it: ss -ltnp | grep 8081
```

Then relaunch. ELI's web port is **8081**, HTTPS **8443**.

Phone can't connect after a restart

Symptom: a paired phone gets 401 / "can't access the server." **Cause:** it's carrying an old token (server restarted, or you rotated the token). **Fix:** the token now **persists** across restarts — just re-open the current link/QR from the **Connect** tab. To deliberately kick a lost phone off, hit **rotate** and re-pair.

Phone can't connect at all (fresh pairing)

Cause: the firewall is blocking the port. **Fix:** run the exact `sudo ufw allow …` lines the server prints on startup, and make sure the phone is on the **same Wi-Fi**.

Phone microphone / voice doesn't work

Cause: browsers block the mic (`getUserMedia`) on a plain `http://LAN-IP` page. **Fix:** start with `--https` and open the `https://…:8443/` link on the phone; accept the one-time self-signed "not private" warning.

Model dropdown in the dashboard is empty

Cause: the list endpoint was being shadowed by the OpenAI-compatible `/v1/models` route. **Fix:** already fixed — the list lives at `/v1/models/installed`. If a fork regresses it, make sure the dashboard fetches that path, not `/v1/models`.

News shows old stories as "the latest"

Cause: the interest-matched half wasn't recency-gated. **Fix:** already fixed (a freshness gate drops stale niche matches). If it recurs, say `refresh the news`.

Terminal shows escape-code junk around a URL (`^[]8;;`…)

Cause: you're looking at a **log file** or `cat -v` output — those *always* render raw escape codes. A live terminal renders the link normally. Not a bug.

`git push` → "Could not resolve host github.com"

Cause: a transient DNS/network blip (ELI is offline-by-default, but git is separate). **Fix:** just retry the push.

Vision / CLIP segfaults on the GPU

Cause: the CLIP/vision path can segfault on some GPUs. **Fix:** CLIP runs on **CPU** by design; keep it there. Main-model + vision hot-swap handle VRAM automatically — don't force CLIP onto the GPU.

Arch / lean distros: "attempt to write a readonly database" (portable install)

Symptom: on the portable Linux build, first-run DB init (Step 5) fails for the `user.*` stores with `sqlite3.OperationalError: attempt to write a readonly database`, while `system_index` / `coding_memory` / `agent` succeed in the same folder; the GUI then can't open the memory DB and exits. **Cause:** the folder you extracted ELI into lives on a filesystem whose locking SQLite's **WAL journal** needs isn't supported — **NTFS, exFAT/FAT, or a network mount** (very common for a `~/Downloads` on a dual-boot data drive). WAL returns `SQLITE_READONLY` there; the rollback-journal stores beside it work fine, which is the tell-tale split. (This is also why the AppImage — which stores data under `~/.local/share/ELI_v2` on your main partition — worked while the portable build in `~/Downloads` didn't.) **Fix:** fixed in **v2.1.19** — ELI detects a WAL-hostile filesystem and falls back to a DELETE (rollback) journal automatically, so it runs wherever you put it. On an older build, extract ELI onto an **ext4/btrfs** path (e.g. under your home partition) instead. Confirm your mount with: `findmnt -T . -o TARGET,FSTYPE`.

Arch / lean distros: GUI won't open — "libxcb-cursor0 is needed to load the Qt xcb platform plugin"

Symptom: the AppImage's engine loads (you see the `ELI v2.0` banner and the module log) but the window never appears; it aborts with the `xcb-cursor0 … xcb platform plugin` message. **Cause:** Qt 6.5+'s xcb platform plugin (`libqxcb.so`) links the **whole xcb-util family** — `libxcb-cursor`, `libxcb-icccm`, `libxcb-image`, `libxcb-keysyms`, `libxcb-render-util`, `libxcb-util` — which a minimal desktop doesn't install. **Qt's message is misleading:** it always blames `libxcb-cursor0` even when the real missing library is a different member (on a test Arch box the actual culprit was `libxcb-icccm.so.4`). **Fix:** fixed in **v2.1.21** — the AppImage now bundles the full xcb-util family (in `PySide6/Qt/lib`), so it launches out of the box on bare Arch/Debian with no extra packages. On an older build, install them yourself:

```
# Arch
sudo pacman -S xcb-util-cursor xcb-util-wm xcb-util-image xcb-util-keysyms xcb-util-renderutil xcb-util
```

```
# Debian/Ubuntu
sudo apt install libxcb-cursor0 libxcb-icccm4 libxcb-image0 libxcb-keysyms1 libxcb-render-util0 libxcb-util1
```

Diagnose exactly which library is missing (Qt's own trace names it):

```
QT_DEBUG_PLUGINS=1 ./ELI_v2-*-x86_64.AppImage 2>&1 | grep -iE 'cannot|not found'
```

Running ELI on Arch — verified working steps

The **AppImage** is the easiest path: it bundles its own **Python 3.11**, so Arch's system Python 3.14 (which has no `llama-cpp-python` wheel) is irrelevant, and as of **v2.1.21** it bundles every Qt xcb library too. Download and run:

```
U=https://github.com/ShadowESC95/ELI_v2.0/releases/download/v2.1.23
wget "$U/ELI_v2-2.1.23-x86_64.AppImage"
chmod +x ELI_v2-2.1.23-x86_64.AppImage
./ELI_v2-2.1.23-x86_64.AppImage
```

Run it directly (as above) so it FUSE-mounts in place — `--appimage-extract-and-run` unpacks ~4 GB into `/tmp`, which fails with `libz.so.1: file too short` when `/tmp` is a small `tmpfs`. If you launch it over **SSH** into a running desktop session, prefix the command with `export DISPLAY=:0`. (Paste long URLs as a **single line** — a wrapped URL 404s and the tail runs as a stray command.)

Verified end-to-end on a clean Arch VM (XFCE): engine loads, database initialises on a WAL-hostile disk via the DELETE fallback, and the GUI opens with **every** xcb-util package uninstalled.

---

*House rule: when a new error bites, add it here — symptom, cause, and the **exact** commands that fixed it. Future-you will thank present-you.*